

What does an AI assistant actually need to be effective?

Group 30
Jonas Lechner, Simon Niederwolfsgruber

ABSTRACT

MiniOpenClaw is a **minimal long-running personal assistant** built to explore what an assistant actually needs in order to be genuinely useful. The project centers on a set of core capabilities: workspace tools, chat-based remote access, durable memory, reusable skills, and proactive background jobs. Tool use is sandboxed by default to keep the system practical as well as safe.

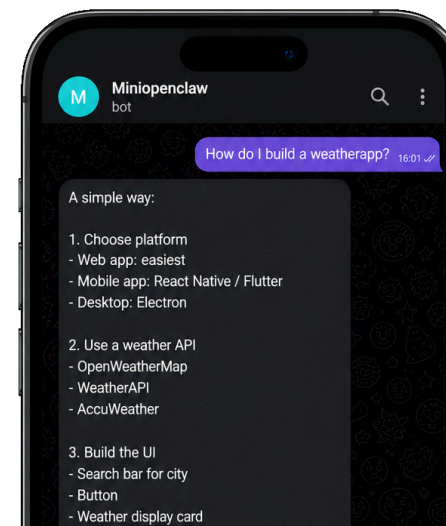
1 Workspace tools

MiniOpenClaw can **read, write, and edit files**, **search** past sessions and documents, **run terminal commands** in a sandbox, and use the web.



2 Chat access

Reach the same ongoing assistant session remotely through Telegram, with built-in commands for session and background-agent management.



3 Memory management

Keep useful context over time without carrying every message forward.

📄 Daily summaries

High-level recap of what happened and why.

⚙️ User & workspace context

Stable facts from onboarding, user notes, and project context files.

🗄️ Session compaction

Reduce context growth by compressing older conversation into smaller state.

🕒 Session history

Past sessions and key outcomes.

4 Skills

Reusable workflows and instructions for recurring tasks, e.g. note capture or interacting with external APIs.

📁 obsidian

Structured note capture and retrieval workflows for connected personal knowledge bases.

🏠 google places

Skill for interacting with the Google Maps API, for example retrieving nearby restaurants with good ratings or public transport information.

...

5 Background jobs & proactivity

Detached runs and scheduled work, kept under user control.

🤖 Subagents

Detached background runs started from Telegram or tool calls, with results fed back into the session.

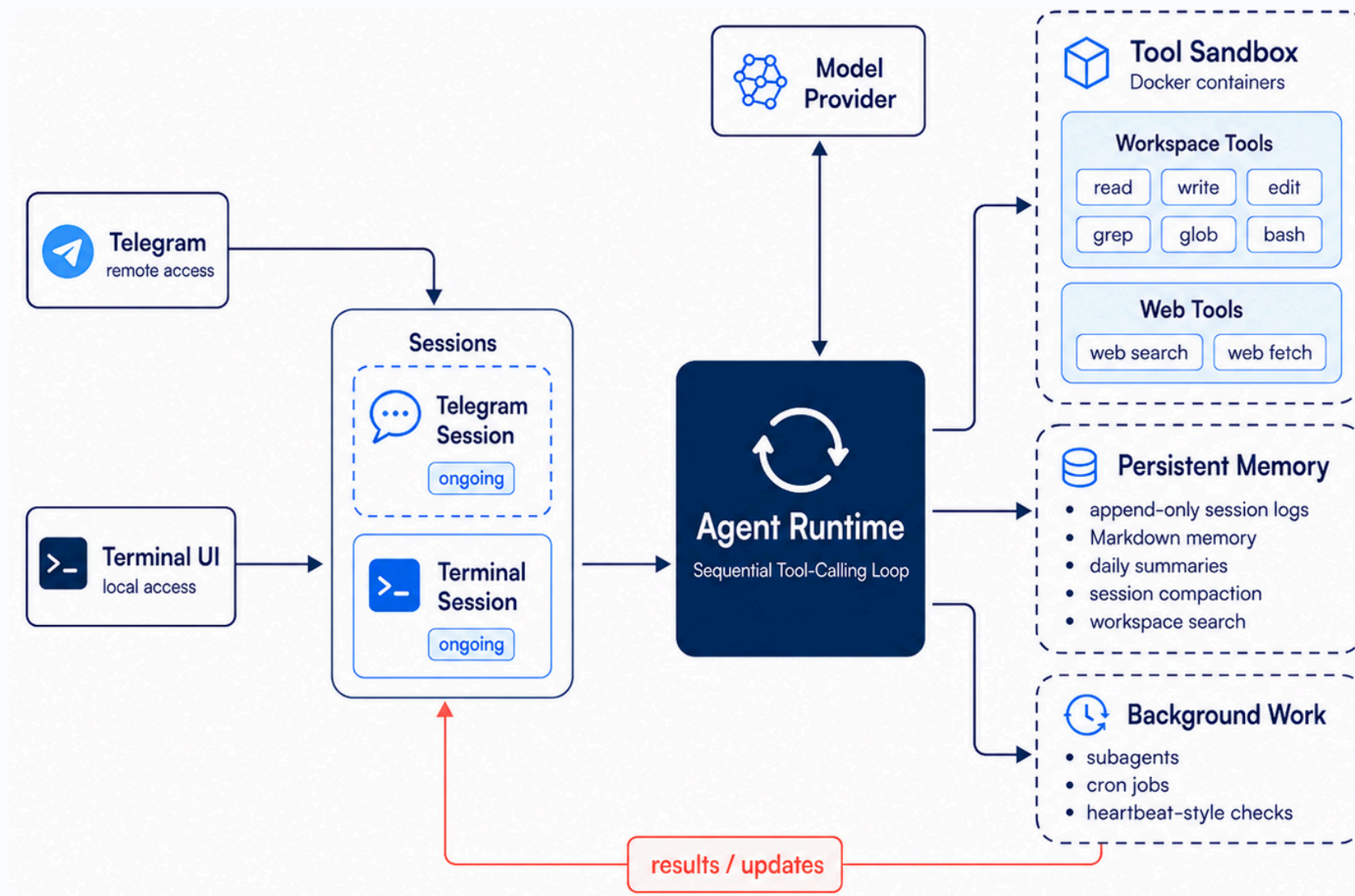
🕒 Cron jobs

Custom recurring tasks such as daily summaries or nightly reflections.

💖 Heartbeat checks

Regular check-ins can revisit open tasks or stale work, for example every 30 minutes.

Architecture



Architecture overview

MiniOpenClaw centers on a **runtime that connects user sessions, model inference, tools, memory, and background work**. It runs a sequential tool-calling loop, supports both Telegram and terminal sessions, queues Telegram messages per chat, streams replies back live, and lets detached jobs send results back into the main session.

Implementation details

We used **pi-ai** for model-provider integration, while the surrounding **runtime and agent loop** were built from scratch. Telegram files are stored in the workspace and their paths are passed to the model automatically, while workspace files can also be returned to the user.

Workspace boundary

All agent sessions are bound to a **workspace** mounted into the **sandbox container**, so **file access**, **search**, and **bash** stay confined to it rather than reaching unrelated parts of the host system.

Sessions & transport

Telegram and **TUI** keep separate current sessions. Each Telegram chat is **bound to one ongoing local session**. **Background work** uses its own fresh detached session and can report results back into the main conversation.

Prompt assembly

The **system prompt** is assembled **once at session start**. It combines **core rules**, **durable context**, **skills**, and **environment metadata**. `user.md` and `context.md` provide persistent human-editable context, while workspace skills are discovered dynamically.

Persistence & compaction

Runs are stored as **append-only session logs**, and **tool events** record arguments, durations, and errors. When history grows too large, **compaction** replaces older messages with a **continuation-focused summary**, using a safe boundary while protecting the current user prompt.

Workspace search

Workspace search provides **cross-session recall** through a local full-text index, so the agent can retrieve relevant notes, memory, and project files without carrying everything in prompt context.

Implementation snapshots & learnings

Gateway Dashboard · Logs

Gateway control

Sessions, tasks, logs, config, restart.

Sessions

Background tasks

Config

Logs

LOCAL CONTROL PLANE

Logs

/home/simonnieder/.mini-openclaw/gateway.log

Logs

/home/simonnieder/.mini-openclaw/gateway.log

18:16:19.000 info gateway_request_completed component=gateway remoteAddress=127.0.0.1

18:16:18.366 info gateway_request_completed component=gateway remoteAddress=127.0.0.1

18:16:16.693 info gateway_request_completed component=gateway

18:16:10.997 error gateway_start_failed component=gateway mess

18:16:10.994 info scheduler_started component=scheduler interv

18:16:10.992 info telegram_polling_started component=telegram

18:16:10.992 info telegram_command_registration_completed comp

Gateway Dashboard · Config

Gateway control

Sessions, tasks, logs, config, restart.

Sessions

Background tasks

Config

Logs

LOCAL CONTROL PLANE

Config

/home/simonnieder/.mini-openclaw/config.json

Config

/home/simonnieder/.mini-openclaw/config.json

{

"gateway": {

"host": "127.0.0.1",

"port": 3000,

"telegram": {

"enabled": true,

"polling": true,

"allowedUserIds": [

"12345"

],

"token": "12345567"

}

}

}

✓ Learnings

- **Up-front planning** mattered more than expected and reduced rework.
- Because AI-assisted work can **move very quickly in different directions**, we needed **more direct communication** to stay aligned, instead of only interacting through our agents.
- The prototype still demonstrated how much can be **engineered in a short time**.
- **/review commands** surfaced many bugs, and we experimented with **CodeRabbit** for AI-assisted code review.

Onboarding

Welcome to MiniOpenClaw. Let's do first-time setup.

What should I call you? Jonas

Should bash command sandboxing be enabled? [Y/n] y

Do you want to set up Telegram now? [y/N] y

Enter your Telegram bot token: 8885885873:AAE807gItYGUg6gehsD_cd

Enter allowed Telegram user ids (comma-separated, optional):

Select an OAuth provider:

1. ChatGPT Plus/Pro (Codex Subscription)

2. GitHub Copilot

Select an option by number: 1

Select a model:

1. gpt-5.1

2. gpt-5.1-codex-max

3. gpt-5.1-codex-mini

4. gpt-5.2

5. gpt-5.2-codex

6. gpt-5.3-codex

7. gpt-5.3-codex-spark

8. gpt-5.4

9. gpt-5.4-mini

10. gpt-5.5

Select an option by number: 8

Authenticating with ChatGPT Plus/Pro (Codex Subscription)...

Please open this URL in your browser to authenticate:

https://auth.openai.com/oauth/authorize?response_type=code&cli

hallenge=eEp08kxs_KDLOVm5gK9v2FLFwpA4r59LmeNpmHQ9AW0&code_challe

A browser window should open. Complete login to finish.

Authentication saved to C:\Users\jonas\.mini-openclaw\auth.json

Onboarding summary:

- Name: Jonas

- Sandbox enabled: true

- Provider: openai-codex

- Model: gpt-5.4

- Telegram enabled: true

Daily summaries as compact memory

Daily summary 2026-06-04

- 2026-06-04

Main work

- Created two new skill scaffolds:

- `skill-pdf-summary` for summarizing PDFs

- `skill-typescript-code-review` for reviewing TypeScript code

- Resolved an initial file creation error by creating the missing skill directory first, then successfully writing `SKILL.md`.

Other activity

- Several test / placeholder / random-text messages.

- Responded to “what’s my name” with `hirete`.

Overall

- Productive day focused mainly on setting up reusable skills; little other substantive work.

! Limitations

- The first memory prototype based on a simple JSON index was **unsuccessful**.
- In practice, only inference via **Copilot and Codex** could be tested extensively.
- **AI subscription limits** / not enough tokens.